

CLAUDE.md – Project Guidelines for Ramify

Project Overview

Ramify (`ramify`) is a stateful session and zero-overhead branching library designed for AI Agents. It provides a lightweight sandbox (via `git worktree` and `COMPOSE_PROJECT_NAME` isolation) allowing agents to safely test, merge, or rollback command side-effects (CWD, ENV, file changes, Docker containers).

Development Commands

- **Install dependencies:** `uv sync` (or `poetry install`)
- **Run tests:** `uv run pytest`
- **Lint & Format:** `uv run ruff check src/ tests/ && uv run ruff format src/ tests/`
- **Type check:** `uv run mypy src/`

Core Architectural Rules

1. **No Custom DSL:** Accept standard Bash string commands in `run()`. Do NOT implement complex operator overloading (`|`, `>`) or custom syntax.
2. **Deterministic Cleanup:** Always ensure resources (Git worktrees, Docker containers via `docker-compose down`, temp dirs) are 100% destroyed on `branch.close()`.
3. **Safety First:** Intercept un-rollbackable host OS commands (`sudo`, `systemctl`, `apt`, `brew`) and raise `GlobalStateError`.
4. **Token Optimization:** Format output into concise JSON (`to_llm_json()`) that strips noise (e.g., long `pip install` logs) to minimize token consumption.
5. **State Persistence:** Maintain `cwd` and `env` across sequential `run()` calls within a session.

Code Style & Conventions

- Python 3.10+ with strict type hints (`from __future__ import annotations`).
- Use `logging` module instead of `print()`.
- Use custom exception hierarchy derived from `RamifyError`.
- Keep functions small, side-effect free where possible, and avoid unnecessary refactoring of working code.

Execution Order

1. **Explore:** Search codebase and understand existing patterns before editing.
2. **Plan:** Propose changes for complex logic before writing code.
3. **Implement & Verify:** Write code and immediately run `pytest` to confirm success.